EDAG01 Course plan 2026

November 1, 2026

Week 1

Read in advance for Lecture 1

- Chapter 1 but you can ignore the Java parts if you have forgotten Java details. The comparisons are a bonus only and you will learn everything anyway.
- Wait with Section 1.4.5 for week 2
- Wait with Section 1.5 for week 2
  - Read Chapter 12 The C Preprocessor: Section 12.1 and stop before start of 12.1.1 and ignore grammar rules if they are confusing
  - Read Section 12.2 but only 12.2.1 and 12.2.2
  - Read Sections 12.3.1 – 12.3.3, and 12.3.7
  - Read Chapter 3 The Power Architecture: Sections 3.1 – 3.2.5 (stop before function calls)

Read in advance for Lecture 2

- Read Appendix B but not details of pseudo code until you need it in a lab (also available as pdf in tresorit)

Recommended short exercises

- 1.2.1
- 1.2.2
- 1.2.7
- 1.4.2
- 1.4.3
- 1.4.5
- 1.4.6

Recommended exercises

- Start on 1.5 RPN for forsete.cs.lth.se

What you should know after week 1

- Enough to start on Lab 1
- Principles of the ISO C language
- Installing and a C compiler on Windows with the Linux Subsystem
- Elementary C programs
- The C proprocessor
- Compiling and executing C programs: link-editor
- Separate compilation
- Arrays in C vs in Java
- Heap memory allocation and deallocation
- Variable length arrays in C99 and C11 (and C18)
- Structs in C vs classes in Java
- Limitations of references in Java
- The need for an explicit pointer in C
- A simple computer model and pointers as integers
- Compiler transformation of parameters with array type into pointer type.
- Array of array memory layout
- How array and pointer expressions are related
- Storage classes
- The nonstandard alloca memory allocation primitive
- Avoiding heap memory allocation and deallocation with free-lists
- The simplex algorithm explained in detail plus pseudo code
- Branch-and-bound for integer linear programming explained in detail plus pseudo code
- Lab and project requirements: 20 integer linear programs should be solved without error

Week 2

Read in advance for Lecture 3

- Read Section 1.4.5 Arrays and matrices allocated in the heap
- Read Section 1.5 Data structures
- Read Chapter 7 sections 7.1 – 7.4 on terminology
- Read Chapter 8 Lexical elements

Read in advance for Lecture 4

- Read Chapter 2 Representation of numbers
- Read Chapter 3 Section 3.2.6 function calls

Recommended short exercises

- 1.4.11
- 1.5.2
- 1.5.3
- 1.5.5

Recommended exercises

- Start on 1.9 FREQ for forsete.cs.lth.se

What you should know after week 2

- Enough to start on Lab 2
- Alternatives for allocating memory for matrices (arrays of arrays)
- Linked lists in C
- Strings in C
- ISO C Standard terminology
- Examples of implementation defined behavior, unspecified behavior and undefined behavior
- Lexical elements and character sets
- Keywords
- Identifiers
- Floating point types and effects on machine code of selecting the proper types
- Computer organization and introduction to RISC and the IBM Power instruction set architecture
- Assembler programs and explaining execution of recursive factorial. Call frame. What is saved there and why.
- From working computer to practical high performance CPUs

Week 3

Read in advance for Lecture 5

- Read Chapter 9 Declarations and stop before Section 9.7 Declarators

Read in advance for Lecture 6

- Read Chapter 3 Sections 3.3 – 3.4 on pipelined and superscalar processors

Recommended short exercises

- 3.4.1
- 3.4.2
- 3.4.3
- 3.4.4
- 3.4.5
- 3.4.6
- 3.4.7

Recommended exercises

- Start on 1.7 POLY for forsete.cs.lth.se

What you should know after week 3

- Enough to start on Lab 3
- The C type system
- Constant expressions
- Recently changed implementation defined behavior of `extern` in gcc and clang
- Differences between `define` and `enum`
- Type constructors: array, pointer-to and function
- Bit-fields and memory locations. Plain int bitfield vs signed or unsigned int bitfield.
- A union larger that its largest field
- Flexible array members and restrictions on their use
- Type qualifiers: const, volatile, restrict
- Dangers with restrict
- Designated initializers
- Computer architecture research: from Zuse 1936 patent on a digital computer to superscalar microarchitecture
- Instruction pipelining. The classic five-stage pipeline and its limitations
- Implementation of out-of-order execution: branch prediction, reorder buffer, rename register
- Reflection of Antoine de Saint-Exupéry on the work of an engineer
- IBM 801 and Power architecture assembler code sequences for high-level code
- Application binary interfaces: what is specifed and function call sequences
- Caller vs callee save register

Read in advance for Lecture 7

- Read Chapter 6 UNIX compilers, debuggers, and profilers
- Read Chapter 15 Writing efficient C code

Read in advance for Lecture 8

- Read the rest of Chapter 7 i.e. Sections 9.7 – 9.12
- Read Chapter 10 Expressions. Most operators are the same as in e.g. Java but be careful with unsigned integer types and especially expressions involving both signed and unsigned integers.

Recommended short exercises

- 4.7.3
- 4.7.4

Recommended exercises

Finish RPN and FREQ.

What you should know after week 4

Week 4

- Enough to start on Lab 4
- Declarations and namespaces
- Declaration vs definition
- More details about variable length arrays vs normal arrays.
- Variably modified types
- Precedence and associativity
- Assignment conversions
- Signed vs unsigned arithmetic and undefined behavior
- Usual arithmetic conversions and integer promotion
- Portable C code
- Compound literals
- Floating point types and expressions. The value NaN
- Pointers and relational operators: valid and invalid expressions
- Valid array optimization and invalid such: let the compiler decide
- Sequence points
- Comma expressions
- Aligning pointers correctly
- Integer divide and remainder in ISO C
- Signed vs unsigned integer arithmetic. Integer type ranks
- Mixing signed and unsigned integer types in expressions. Warnings!
- Cache memories
- Temporal and spatial locality
- Cache associativity
- Cache block size
- Exploiting cache memories

Read in advance for Lecture 9

- Read Chapter 4 Cache memories

Read in advance for Lecture 10

- Read Chapter 11 Statements
- Read the rest of Chapter 12

Recommended short exercises

- 12.1.1
- 12.1.2
- 12.1.3
- 12.1.4

Recommended exercises

Finish POLY.

What you should know after week 5

Week 5

- Enough to start on Lab 5
- More details about the C preprocessor (all directives)
- Stringizer operator
- Token concatenation operator
- Function-like macros with variable number of arguments
- Problems with macros
- Problems with inline functions
- Restrictions of inline functions and why these are needed
- Semantic differences of gcc inline and ISO C inline and how to overcome it in a portable way
- Let the compiler decide
- Purpose of the do-while (0) macro
- Statements. Most is trivial but new rules in C11 about which loops can be optimized away
- Premature optimization is the root of all evil (Knuth)
- First correct and then fast
- A methodology to optimize C code manually
- Purpose of reference implementations
- Our definition of implementation tuning: what current compilers cannot do automatically
- Making the common case fast
- Profiling tools: operf, gprof, gcov, printf, cachegrind
- Performance monitor counters
- Using simulators for performance measurements. The IBM Power8 pipeline simulator and visualizer

Read in advance for Lecture 11

- Read the first page of Chapter 13 the C Standard Library and about the following header files:

  ```
  <assert.h>
  <ctype.h>
  <fenv.h>
  <errno.h>
  <math.h>
  <stdint.h>
  <inttypes.h>
  <setjmp.h>
  <signal.h>
  <string.h>
  <stdlib.h>
  <stdio.h>
  ```

  You don’t need to know every detail about these header files but the examples in the book are useful.

Read in advance for Lecture 12

- Read Chapter 16 Optimizing compilers

Recommended short exercises

- 15.6.1

Recommended exercises

Try to improve the scores of RPN, FREQ, POLY, and INTOPT and reflect on why it worked or not.

What you should know after week 6

Week 6

- Enough to start on Lab 6
- The ISO C Standard library overview
- Math functions
- setjmp and longjmp
- signals
- String functions
- Safety critical code and avoiding buffer overflows
- C vs C++ and the problems of using the full C++ language as observed at Google and why they don’t use the full language
- Number of people in the world who know basically everything about C vs about C++
- When and why I use C++ and why C is my favorite language since 1988
- How C compilers are validated for ISO compliance
- Manual code tuning vs automatic by optimizing compilers
- We need to understand what compilers can do to write C code that is easier to optimize

- Overview of optimizing compilers
- Dominance analysis with Lengauer-Tarjan algorithm
- Loop analysis
- SSA Form
- Global value numbering
- Partial redundancy elimination
- Operator strength reduction
- Constant propagation
- Dead code elimination
- Instruction scheduling and modulo scheduling
- Register allocation